(U) T-150/TRV-150 Heavy-Lift UAS
Architecture Description Document



Prepared By: [PLACEHOLDER – Preparing Organization]
Revision: 3.0
Date: 20260305

Controlled by:	[PLACEHOLDER]
Distribution/Dissemination Controls:	[PLACEHOLDER – Distribution Statement]
POC:	[PLACEHOLDER – Point of Contact]
Contact:	[PLACEHOLDER]@mail.mil

UNCLASSIFIED			
Revision	Date	Author	Description of Changes
1.0	20250615	[PLACEHOLDER]	Initial draft – battery, propulsion, VRS processors
2.0	20251001	[PLACEHOLDER]	Added slung load processor, 6DOF mover, inter-plugin communication
3.0	20260305	[PLACEHOLDER]	Complete architecture description, UML diagrams, design decisions
			UNCLASSIFIED

(U) Table of Contents

1 (U) Introduction 1

1.1 (U) Purpose . . . 1

1.2 (U) Scope . . . 1

1.3 (U) Threat System Description . . . 1

1.4 (U) Simulation Framework . . . 2

1.5 (U) Design Standards . . . 2

2 (U) Model Architecture Description 3

2.1 (U) Architecture Overview . . . 3

2.2 (U) UML Class Diagram . . . 5

2.3 (U) Plugin Registration Pattern . . . 6

2.4 (U) Inter-Processor Communication via AuxData . . . 7

2.4.1 (U) Mover (Rotorcraft6DOF) – Published Keys . . . 7

2.4.2 (U) Mover (Rotorcraft6DOF) – Read Keys . . . 8

2.4.3 (U) BatteryProcessor – Published Keys . . . 8

2.4.4 (U) BatteryProcessor – Read Keys . . . 9

2.4.5 (U) PropulsionProcessor – Published Keys . . . 9

2.4.6 (U) VRSProcessor – Published Keys . . . 10

2.4.7 (U) SlungLoadProcessor – Published Keys . . . 10

3 (U) Component: Rotorcraft6DOF Mover 11

3.1 (U) Structural Overview . . . 11

3.2 (U) Composition Structure . . . 11

3.3 (U) WsfRouteMover Interface . . . 12

3.4 (U) UpdatePosition Sequence . . . 12

3.5 (U) Spline Navigation State Machine . . . 13

3.6 (U) Rotor Configuration System . . . 14

3.7 (U) Public Interface Summary . . . 15

4 (U) Component: BatteryProcessor 16

4.1 (U) Structural Overview . . . 16
4.2 (U) Lifecycle . . . 17
4.3 (U) Power Source Resolution . . . 18
4.4 (U) Voltage Sag Solver . . . 18
4.5 (U) Public Interface Summary . . . 19
5 (U) Component: PropulsionProcessor 20
5.1 (U) Structural Overview . . . 20
5.2 (U) Motor State Array . . . 20
5.3 (U) Update Loop . . . 21
5.4 (U) MotorState Machine . . . 22
5.5 (U) Public Interface Summary . . . 22
6 (U) Component: VRSProcessor 23
6.1 (U) Structural Overview . . . 23
6.2 (U) VRS State Machine . . . 23
6.3 (U) Velocity Reading . . . 24
6.4 (U) Public Interface Summary . . . 24
7 (U) Component: SlungLoadProcessor 25
7.1 (U) Structural Overview . . . 25
7.2 (U) Pendulum Integration Loop . . . 25
7.3 (U) SlungLoad State Machine . . . 26
7.4 (U) Force Coupling to Mover . . . 27
7.5 (U) Public Interface Summary . . . 27
8 (U) Design Decisions 28
8.1 (U) Decision Log . . . 28
9 (U) Installation and Build 30
9.1 (U) Build System . . . 30

9.2 (U) CMake Configuration 31
9.3 (U) Build and Install Scripts 32
9.4 (U) Compiler Requirements 33
10(U) Sources and References 34
11(U) Acronyms 35

(U) List of Tables

1 (U) Plugin Suite Summary . . . 1
2 (U) Mover Published AuxData Keys . . . 7
3 (U) Mover Read AuxData Keys . . . 8
4 (U) Battery Published AuxData Keys . . . 8
5 (U) Battery Read AuxData Keys . . . 9
6 (U) Propulsion Published AuxData Keys . . . 9
7 (U) VRS Published AuxData Keys . . . 10
8 (U) Slung Load Published AuxData Keys . . . 10
9 (U) Rotorcraft6DOF Source Files . . . 11
10 (U) WsfRouteMover Interface Methods . . . 12
11 (U) Spline Navigation States . . . 14
12 (U) Rotor Configurations . . . 14
13 (U) Rotorcraft6DOF Public Interface . . . 15
14 (U) Battery Source Files . . . 16
15 (U) BatteryProcessor Public Interface . . . 19
16 (U) Propulsion Source Files . . . 20
17 (U) PropulsionProcessor Public Interface . . . 22
18 (U) VRS Source Files . . . 23
19 (U) VRS State Descriptions . . . 23
20 (U) VRSProcessor Public Interface . . . 24
21 (U) Slung Load Source Files . . . 25
22 (U) Slung Load State Descriptions . . . 26
23 (U) SlungLoadProcessor Public Interface . . . 27
24 (U) Design Decision Log . . . 28
25 (U) Build and Install Scripts . . . 32
26 (U) Install Destinations . . . 33
27 (U) Compiler Requirements . . . 33
28 (U) References . . . 34
29 (U) Acronyms . . . 35

(U) List of Figures

1 (U) Architecture Overview . . . 4
2 (U) UML Class Hierarchy . . . 5
3 (U) Plugin Registration Sequence . . . 6
4 (U) UpdatePosition Sequence . . . 13
5 (U) Spline Navigation State Machine . . . 13
6 (U) Battery Processor Lifecycle Sequence . . . 17
7 (U) Power Source Resolution . . . 18
8 (U) Motor State Array . . . 21
9 (U) Propulsion Update Loop . . . 21
10 (U) Motor State Machine . . . 22
11 (U) VRS State Machine . . . 23
12 (U) Slung Load Pendulum Integration Loop . . . 26
13 (U) Slung Load State Machine . . . 26
14 (U) Build Directory Structure . . . 31

1 (U) Introduction

1.1 (U) Purpose

(U) This Architecture Description Document (ADD) defines the software architecture of the T-150 UAS AFSIM Plugin Suite. The document describes the code structure, class hierarchies, inter-component communication mechanisms, plugin registration patterns, and design decisions that govern the implementation. This ADD is intended to satisfy the architecture documentation requirements of DoDI 5000.61 **DoDI500061** and MIL-STD-3022 **MILSTD3022** for Modeling and Simulation accreditation.

(U) This document focuses exclusively on software architecture. The underlying physics algorithms, mathematical formulations, and validation data are documented separately in the Underlying Algorithms Model (UAM) **T150UAM**.

1.2 (U) Scope

(U) The T-150 Plugin Suite comprises five AFSIM plugins (six shared libraries) that collectively model a coaxial octocopter UAS within the Advanced Framework for Simulation, Integration, and Modeling (AFSIM) environment. The suite totals approximately 7,736 lines of C++ source across 26 files.

(U) The plugins are:

Table 1: (U) Plugin Suite Summary

UNCLASSIFIED			
Plugin	Shared Library	Type Registered	LOC
Rotorcraft 6DOF Mover	rotorcraft_6dof	WSF_ROTORCRAFT_ 6DOF_MOVER	2,939
Battery Processor	t150_battery_ processor	T150_BATTERY_ PROCESSOR	1,139
Propulsion Processor	t150_propulsion	T150_PROPULSION_ PROCESSOR	1,281
VRS Processor	t150_vrs	T150_VRS_PROCESSOR	697
Slung Load Processor	t150_slung_load	T150_SLUNG_LOAD_ PROCESSOR	825
			UNCLASSIFIED

1.3 (U) Threat System Description

(U) The T-150 is a heavy-lift coaxial octocopter UAS with 8 motors on four arms in a coaxial configuration (two motors per arm, counter-rotating). It is designed for cargo delivery and slung-load transport missions in contested environments. Further system description is provided in **T150UAM**.

1.4 (U) Simulation Framework

(U) The plugins are developed for AFSIM version 2.2505. They are compiled as shared libraries (.so on Linux, .dll on Windows) and loaded at runtime by the AFSIM plugin loader. The codebase requires C++17 or later (CMake is configured for C++20) and is compiled with GCC 13.3 / MSVC on the target platforms.

(U) AFSIM provides the following base classes that the plugins extend:

- (U) **WsfRouteMover** – Base class for movers that follow routes composed of waypoints. Inherits from WsfMover.
- (U) **WsfProcessor** – Base class for processors that attach to platforms and execute per-timestep logic.
- (U) **WsfPlatform** – Represents a simulation entity. Hosts mover, processors, sensors, and an AuxData attribute store.
- (U) **WsfScenario** – Top-level container for all simulation entities and types.

1.5 (U) Design Standards

- (U) All plugins conform to the AFSIM plugin API defined by WsfPlugin.hpp.
- (U) Inter-plugin communication uses the AuxData key-value store on the shared WsfPlatform, avoiding direct coupling between plugins.
- (U) Each plugin is independently deployable; fallback values ensure graceful degradation when dependencies are unavailable.
- (U) No global mutable state is shared between plugins beyond the AuxData mechanism.
- (U) Naming conventions follow AFSIM patterns: PascalCase for classes, mCamelCase for member variables, UPPER_SNAKE_CASE for AuxData keys.

2 (U) Model Architecture Description

2.1 (U) Architecture Overview

(U) The T-150 Plugin Suite follows a loosely-coupled component architecture. Each plugin is compiled into a separate shared library and loaded independently by AFSIM at runtime. The plugins communicate exclusively through the platform's AuxData key-value store, which acts as a shared blackboard. No plugin holds a direct reference to any other plugin's classes.

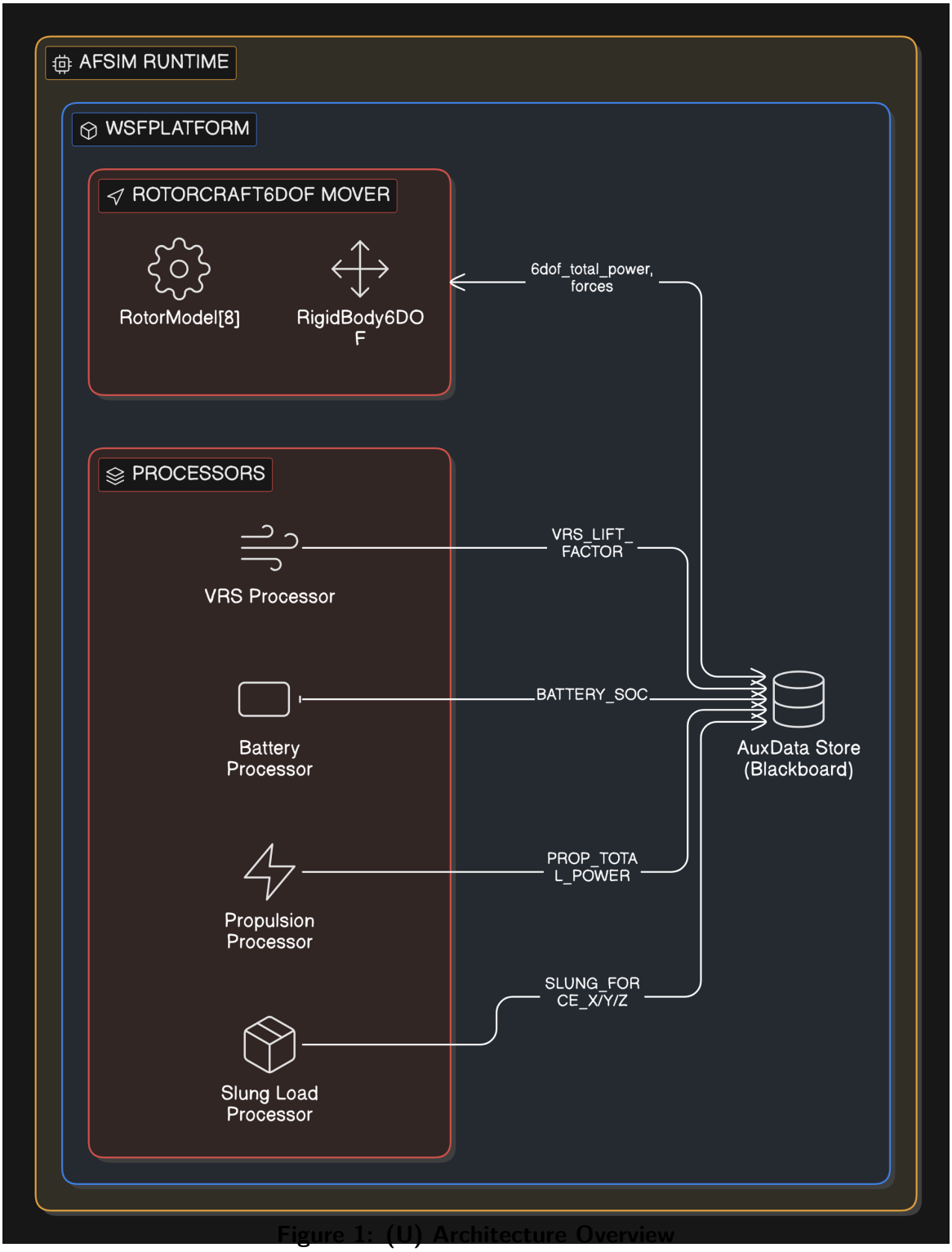


Figure 1: (U) Architecture Overview

(U) **Data flow direction**: Each processor writes its outputs to AuxData and reads its inputs from AuxData. The mover writes telemetry (position, velocity, thrust, power) and reads external forces (slung load, weapon recoil). The battery processor reads power from the propulsion processor or mover. This blackboard pattern ensures that any subset of plugins can operate without the others present.

2.2 (U) UML Class Diagram

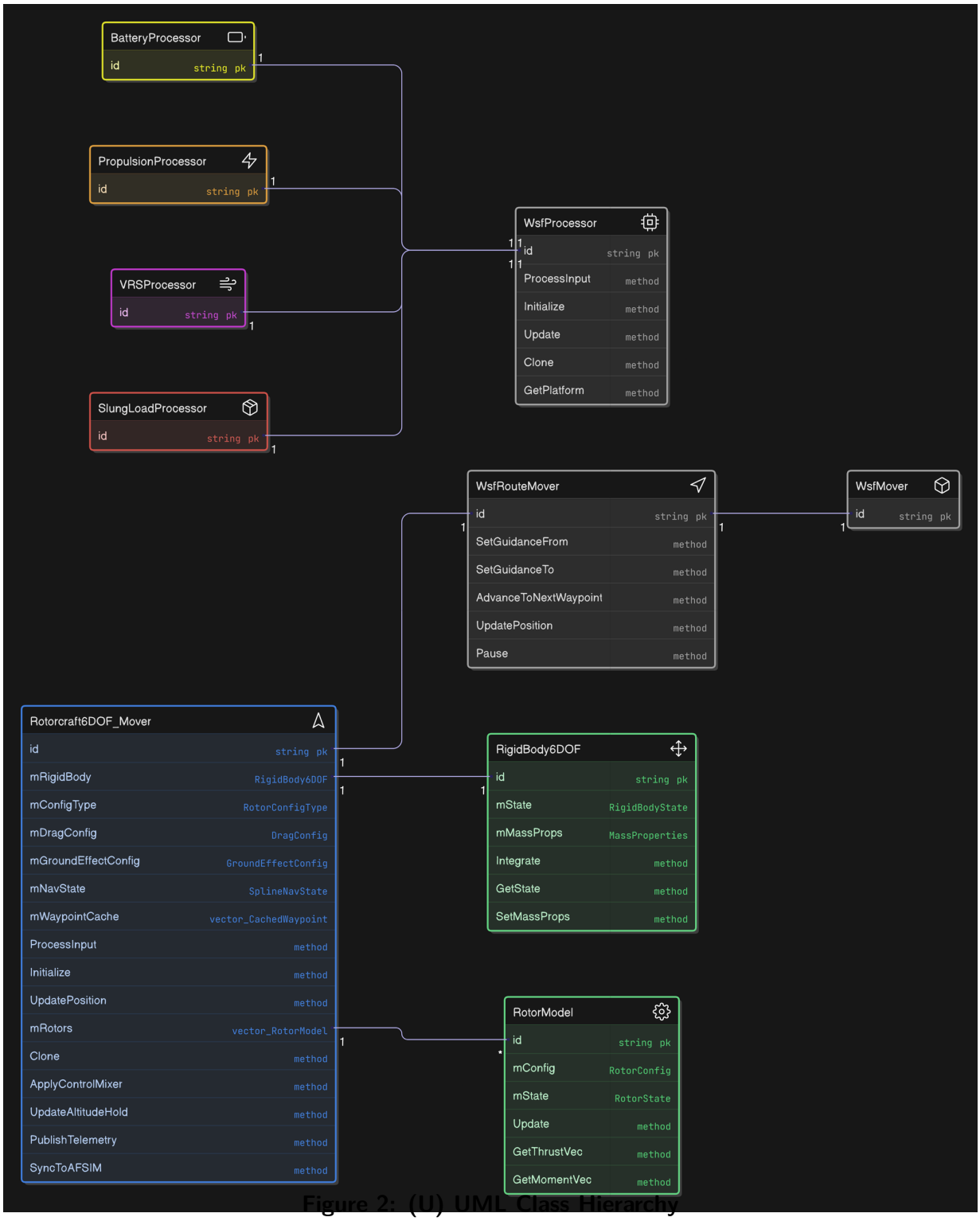


Figure 2: (U) UML Class Hierarchy

2.3 (U) Plugin Registration Pattern

(U) All five plugins follow the same two-level registration pattern to register their types with AFSIM. This is the standard AFSIM plugin API contract.

(U) **Registration sequence (pseudocode):**

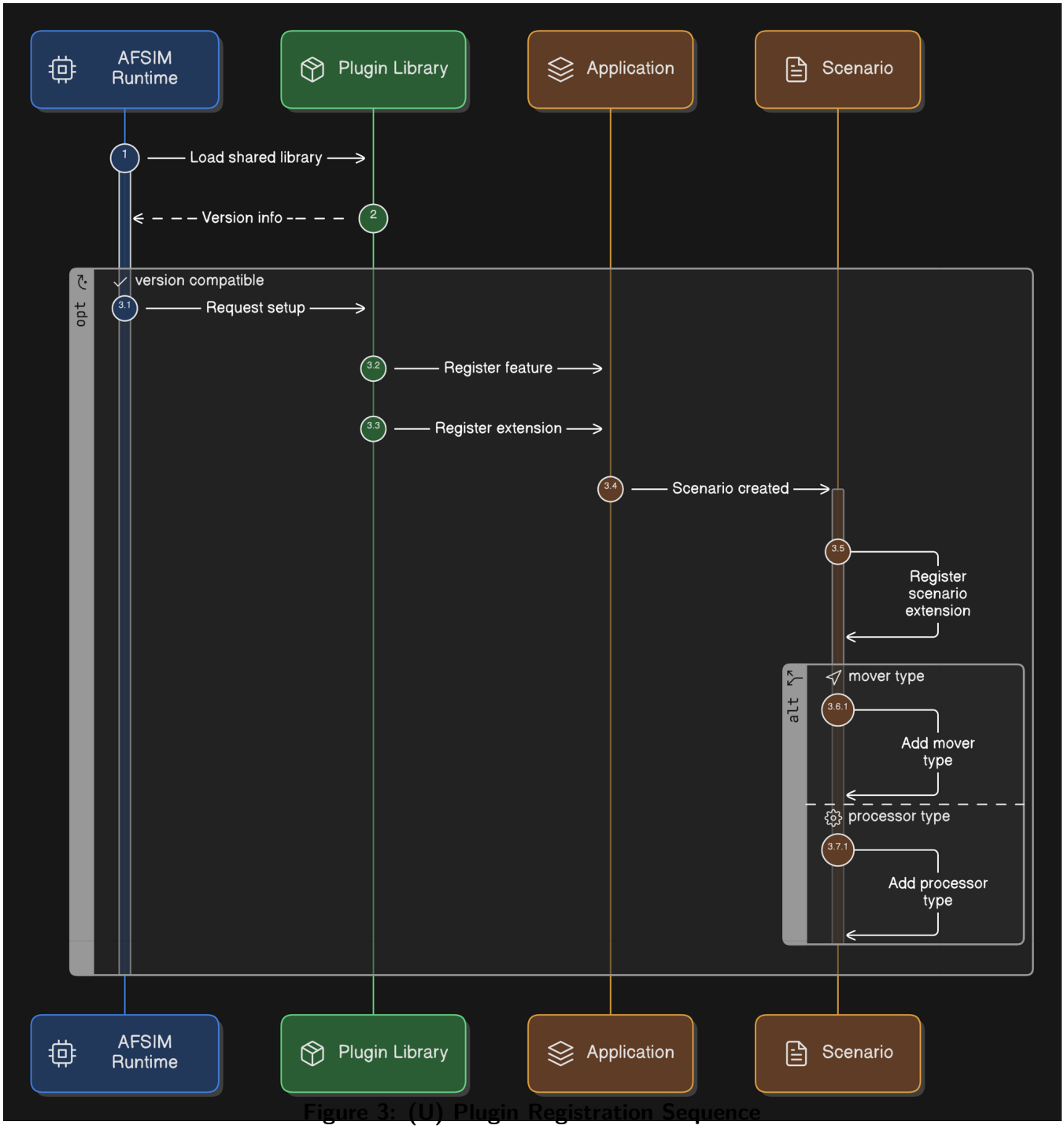


Figure 3: (U) Plugin Registration Sequence

(U) **Preconditions**: AFSIM must be initialized and the plugin shared library must be locatable via the plugin search path or scenario plugin_directory directive.

(U) **Expected outcomes**: After registration, the type name (e.g., WSF_ROTORCRAFT_6DOF_MOVER or T150_BATTERY_PROCESSOR) becomes available for use in scenario files.

(U) The 6DOF mover plugin uses the WsfDefaultApplicationExtension template helper, while the four processor plugins define a custom ApplicationExtension class with an identical pattern. Both approaches produce the same result: a ScenarioExtension is attached to each new scenario to register the plugin's type.

2.4 (U) Inter-Processor Communication via AuxData

(U) Plugins communicate through the platform's WsfAttributeContainer (AuxData). Each plugin writes its outputs as named key-value pairs and reads inputs from keys published by other plugins. The following tables catalog all AuxData keys in the suite.

2.4.1 (U) Mover (Rotorcraft6DOF) – Published Keys

Table 2: (U) Mover Published AuxData Keys

UNCLASSIFIED		
Key	Type	Description
6dof_pos_n, _e, _d	double	NED position [m]
6dof_vel_n, _e, _d	double	NED velocity [m/s]
6dof_roll_deg, _pitch_deg, _yaw_deg	double	Euler angles [deg]
6dof_p_dps, _q_dps, _r_dps	double	Angular rates [deg/s]
6dof_total_thrust	double	Sum of all rotor thrust [N]
6dof_total_power	double	Sum of all rotor power [W]
6dof_throttle_cmd	double	Throttle command [0–1]
6dof_avg_rpm	double	Average rotor RPM
6dof_mass, 6dof_weight	double	Current mass [kg], weight [N]
		UNCLASSIFIED

2.4.2 (U) Mover (Rotorcraft6DOF) – Read Keys

Table 3: (U) Mover Read AuxData Keys

UNCLASSIFIED		
Key	Source Plugin	Description
SLUNG_FORCE_X/Y/Z	SlungLoadProcessor	Cable reaction forces [N]
WEAPON_FORCE_X/Y/Z	WeaponProcessor	Recoil forces [N]
WEAPON_MOMENT_X/Y/Z	WeaponProcessor	Recoil moments [Nm]
WEAPON_REGISTRY	WeaponProcessor	Comma-separated weapon names
WEAPON_<name>_MASS_DELTA	WeaponProcessor	Ammunition mass change [kg]
SLUNG_LOAD_STATE	SlungLoadProcessor	Load state enumeration
SLUNG_LOAD_MASS	SlungLoadProcessor	Load mass for dynamic mass update [kg]
		UNCLASSIFIED

2.4.3 (U) BatteryProcessor – Published Keys

Table 4: (U) Battery Published AuxData Keys

UNCLASSIFIED		
Key	Type	Description
BATTERY_SOC	double	State of charge [0–1]
BATTERY_POWER_W	double	Current power draw [W]
BATTERY_ENERGY_WH	double	Remaining energy [Wh]
BATTERY_VOLTAGE	double	Terminal voltage [V]
BATTERY_OCV	double	Open-circuit voltage [V]
BATTERY_VOLTAGE_SAG	double	Voltage drop from internal resistance [V]
BATTERY_CURRENT_A	double	Pack current [A]
BATTERY_LOW	bool	Low SOC warning flag
BATTERY_VOLTAGE_LOW	bool	Voltage sag warning flag
		UNCLASSIFIED

2.4.4 (U) BatteryProcessor – Read Keys

Table 5: (U) Battery Read AuxData Keys

UNCLASSIFIED		
Key	Source Plugin	Description
PROP_TOTAL_POWER	PropulsionProcessor	Propulsion power [W] (primary source)
6dof_total_power	Rotorcraft6DOF Mover	Mover power [W] (fallback source)
		UNCLASSIFIED

2.4.5 (U) PropulsionProcessor – Published Keys

Table 6: (U) Propulsion Published AuxData Keys

UNCLASSIFIED		
Key	Type	Description
PROP_TOTAL_THRUST	double	Total thrust [N]
PROP_TOTAL_POWER	double	Total electrical power [W]
PROP_SYSTEM_EFF	double	System efficiency [0–1]
PROP_MOTOR_COUNT	int	Operational motor count
PROP_GROUND_EFFECT	double	Ground effect multiplier
PROP_ALTITUDE_AGL	double	Altitude AGL [m]
PROP_TOTAL_MASS	double	Total mass [kg]
PROP_PAYLOAD_MASS	double	Payload mass [kg]
PROP_THRUST_MARGIN	double	Available thrust margin [0–1]
PROP_CG_OFFSET_X/Y/Z	double	Combined CG offset [m]
PROP_FORWARD_SPEED	double	Horizontal speed [m/s]
PROP_CLIMB_RATE	double	Vertical speed [m/s]
PROP_BANK_ANGLE	double	Bank angle [deg]
PROP_OVERLOADED	bool	Payload exceeds capacity
MOTOR_OVERTEMP	bool	Motor over-temperature flag
MOTOR_FAILURE	bool	Motor failure flag
		UNCLASSIFIED

2.4.6 (U) VRSProcessor – Published Keys

Table 7: (U) VRS Published AuxData Keys

UNCLASSIFIED		
Key	Type	Description
VRS_MARGIN	double	VRS proximity margin [0–1]
VRS_STATE	int	VRS state enumeration
VRS_LIFT_FACTOR	double	Lift degradation factor [0–1]
VRS_CONTROL_FACTOR	double	Control authority factor [0–1]
VRS_DEVELOPMENT	double	VRS development level [0–1]
VRS_DESCENT_RATE	double	Current descent rate [m/s]
VRS_FORWARD_SPEED	double	Current forward speed [m/s]
VRS_CAUTION, _ONSET, _CRITICAL	bool	Warning flags
		UNCLASSIFIED

2.4.7 (U) SlungLoadProcessor – Published Keys

Table 8: (U) Slung Load Published AuxData Keys

UNCLASSIFIED		
Key	Type	Description
SLUNG_LOAD_STATE	int	Load state enumeration
SLUNG_LOAD_MASS	double	Load mass [kg]
SLUNG_CABLE_LENGTH	double	Cable length [m]
SLUNG_CABLE_TENSION	double	Cable tension [N]
SLUNG_SWING_X, _Y	double	Swing angles [deg]
SLUNG_SWING_MAG	double	Swing magnitude [deg]
SLUNG_FORCE_X/Y/Z	double	Reaction forces on aircraft [N]
SLUNG_POS_X/Y/Z	double	Load position relative to CG [m]
SLUNG_RESONANCE	bool	Resonance warning flag
SLUNG_NATURAL_FREQ	double	Pendulum natural frequency [Hz]
		UNCLASSIFIED

3 (U) Component: Rotorcraft6DOF Mover

3.1 (U) Structural Overview

(U) The Rotorcraft6DOF Mover is the core flight dynamics component. It extends WsfRouteMover (which itself extends WsfMover) to inherit route/waypoint navigation integration while replacing the position update logic with full 6DOF physics.

(U) **Source files:**

Table 9: (U) Rotorcraft6DOF Source Files

UNCLASSIFIED		
File	Purpose	LOC
Rotorcraft6DOF_Mover.hpp	Class declaration, enums, structs	489
Rotorcraft6DOF_Mover.cpp	Implementation	∼2,450
RigidBody6DOF.hpp	Rigid body state and integrator	214
RigidBody6DOF.cpp	Integration implementation	∼150
RotorModel.hpp	Single rotor model	181
RotorModel.cpp	Rotor physics implementation	∼120
MathUtils.hpp	Vector3, Quaternion, Matrix3x3, spline math	511
Rotorcraft6DOF_Registration.hpp	Scenario extension	56
Rotorcraft6DOF_ PluginRegistration.cpp	Plugin entry points	57
		UNCLASSIFIED

(U) **Namespace**: rotorcraft_6dof

3.2 (U) Composition Structure

(U) The mover contains the following owned objects:

- (U) **RigidBody6DOF** (mRigidBody) – Holds the kinematic state (position, velocity, orientation quaternion, angular velocity) and mass properties. Provides the Integrate() method that advances state by one timestep given total forces and moments.
- (U) **vector<RotorModel>** (mRotors) – Array of N rotor models, sized by configuration (4 for QUAD, 8 for COAXIAL_QUAD). Each rotor independently computes thrust, torque, and power from its commanded RPM.
- (U) **DragConfig** (mDragConfig) – Aerodynamic drag coefficients and reference area.

- (U) **GroundEffectConfig** (mGroundEffectConfig) – Ground effect parameters.
- (U) **map<int, WaypointConfig>** (mWaypointConfigs) – Per-waypoint behavior overrides for spline navigation.
- (U) **vector<CachedWaypoint>** (mWaypointCache) – Sliding window of four waypoints for Catmull-Rom spline interpolation.

3.3 (U) WsfRouteMover Interface

(U) The mover implements the pure virtual methods required by WsfRouteMover:

Table 10: (U) WsfRouteMover Interface Methods

UNCLASSIFIED	
Method	Behavior
SetGuidanceFrom(waypoint)	Caches the previous waypoint for speed reference.
SetGuidanceTo(waypoint)	Sets target altitude and speed from the next waypoint.
AdvanceToNextWaypoint(time, target, next)	Configures guidance for the next segment. If spline navigation is enabled, advances the waypoint cache and recomputes the speed profile.
UpdatePosition(simTime)	Core physics loop – computes forces, integrates rigid body, synchronizes AFSIM platform state.
Pause(simTime, unpauseTime)	Enters hover-hold at current position until unpauseTime.
	UNCLASSIFIED

3.4 (U) UpdatePosition Sequence

(U) The UpdatePosition() method executes the following sequence each timestep:

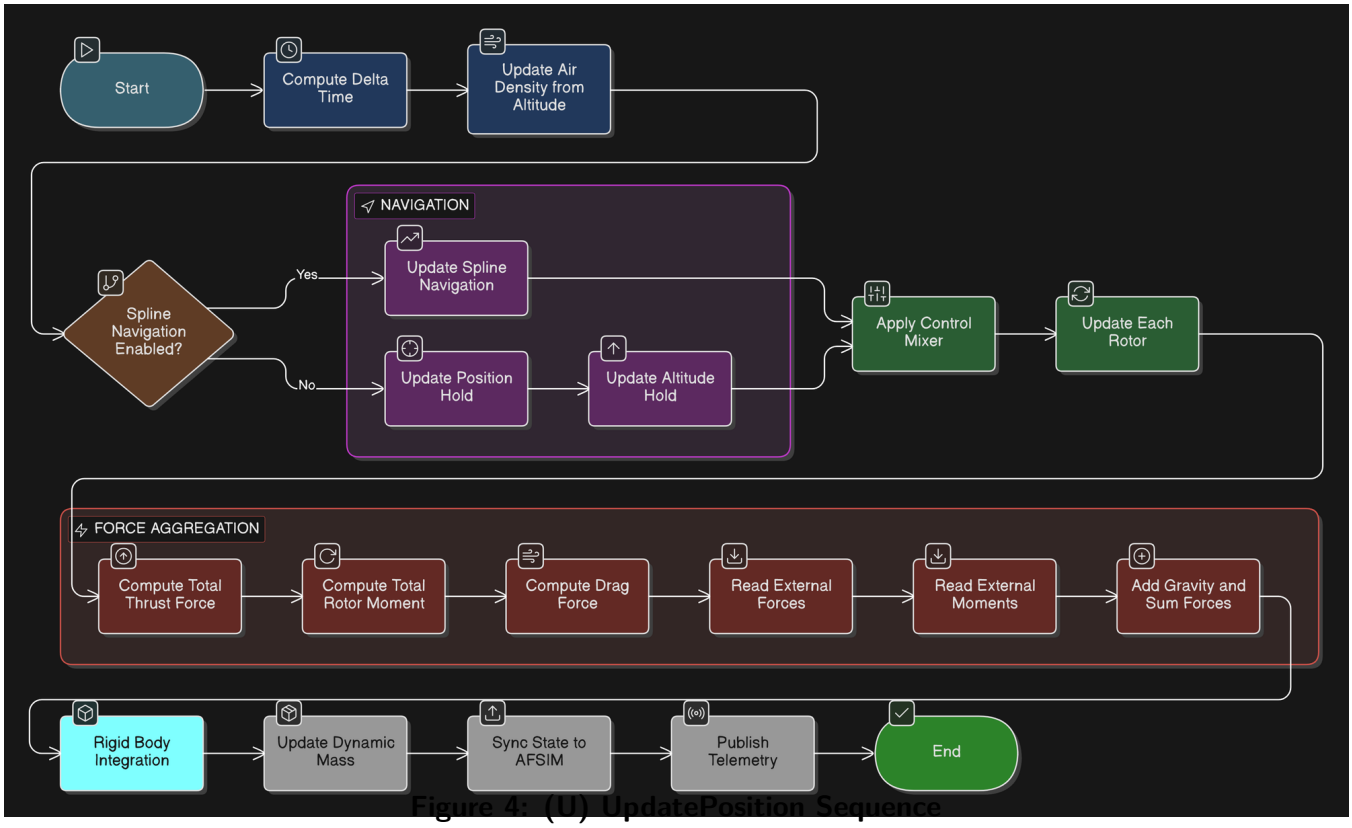


Figure 4: (U) UpdatePosition Sequence

3.5 (U) Spline Navigation State Machine

(U) When spline navigation is enabled, the mover uses a Catmull-Rom spline interpolation across a sliding window of four cached waypoints (P0, P1, P2, P3) to produce smooth flight paths. The navigation state machine governs behavior at each waypoint.

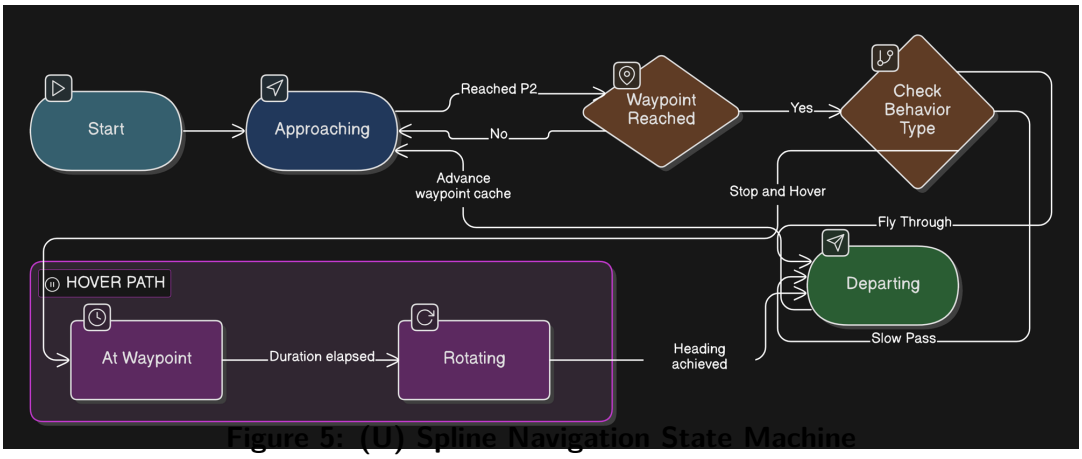


Figure 5: (U) Spline Navigation State Machine

(U) **State descriptions:**

Table 11: (U) Spline Navigation States

UNCLASSIFIED			
State	Precondition	Behavior	Exit Condition
APPROACHING	Waypoint cache initialized	Follow spline toward P2 at profiled speed	Distance to P2 $<$ arrival radius
AT_WAYPOINT	STOP_AND_HOVER behavior	Hold position, zero velocity	Hover duration elapsed
ROTATING	Departure heading specified	Yaw to face next waypoint	Heading error $<$ threshold
DEPARTING	Rotation complete or FLY_THROUGH	Advance waypoint cache, accelerate	Spline parameter reset, transition to APPROACHING
			UNCLASSIFIED

3.6 (U) Rotor Configuration System

(U) The mover supports multiple rotor configurations through the RotorConfigType enumeration:

Table 12: (U) Rotor Configurations

UNCLASSIFIED			
Configuration	Rotors	Arms	Description
QUAD_X	4	4	X-pattern quadcopter
QUAD_PLUS	4	4	Plus-pattern quadcopter
HEX_X	6	6	Hexacopter
OCTO_X	8	8	Flat octocopter
COAXIAL_QUAD	8	4	Coaxial quadcopter (T-150 default)
COAXIAL_OCTO	16	8	Coaxial octocopter
CUSTOM	N	N	User-defined rotor positions
			UNCLASSIFIED

(U) Each Setup*() method populates the mRotors vector with RotorModel instances whose RotorConfig specifies body-frame position, rotation direction (CW/CCW), thrust axis, and control mixing factors (roll, pitch, yaw).

3.7 (U) Public Interface Summary

Table 13: (U) Rotorcraft6DOF Public Interface

UNCLASSIFIED		
Method	Precondition	Expected Outcome
ProcessInput(input)	Scenario file parsed	Configuration loaded: mass, rotors, gains, spline settings
Initialize(simTime)	ProcessInput complete	6DOF state initialized from AFSIM platform position, hover RPM computed
UpdatePosition(simTime)	Initialize complete	Platform position/velocity/attitude advanced by one timestep
SetRotorConfigType(type)	Before Initialize	Rotor array populated with preset geometry
GetTotalThrust()	Any time after Initialize	Returns sum of all rotor thrust values [N]
GetTotalPower()	Any time after Initialize	Returns sum of all rotor power values [W]
		UNCLASSIFIED

4 (U) Component: BatteryProcessor

4.1 (U) Structural Overview

(U) The BatteryProcessor models a lithium-ion battery pack, tracking state-of-charge (SOC), terminal voltage under load, and energy consumption. It extends WsfProcessor and follows the standard ProcessInput/Initialize/Update lifecycle.

(U) **Source files:**

Table 14: (U) Battery Source Files

UNCLASSIFIED	
File	Purpose
BatteryProcessor.hpp	Class declaration
BatteryProcessor.cpp	Implementation
BatteryRegistration.hpp	Scenario extension
BatteryPluginRegistration.cpp	Plugin entry points
	UNCLASSIFIED

(U) **Namespace**: t150

4.2 (U) Lifecycle

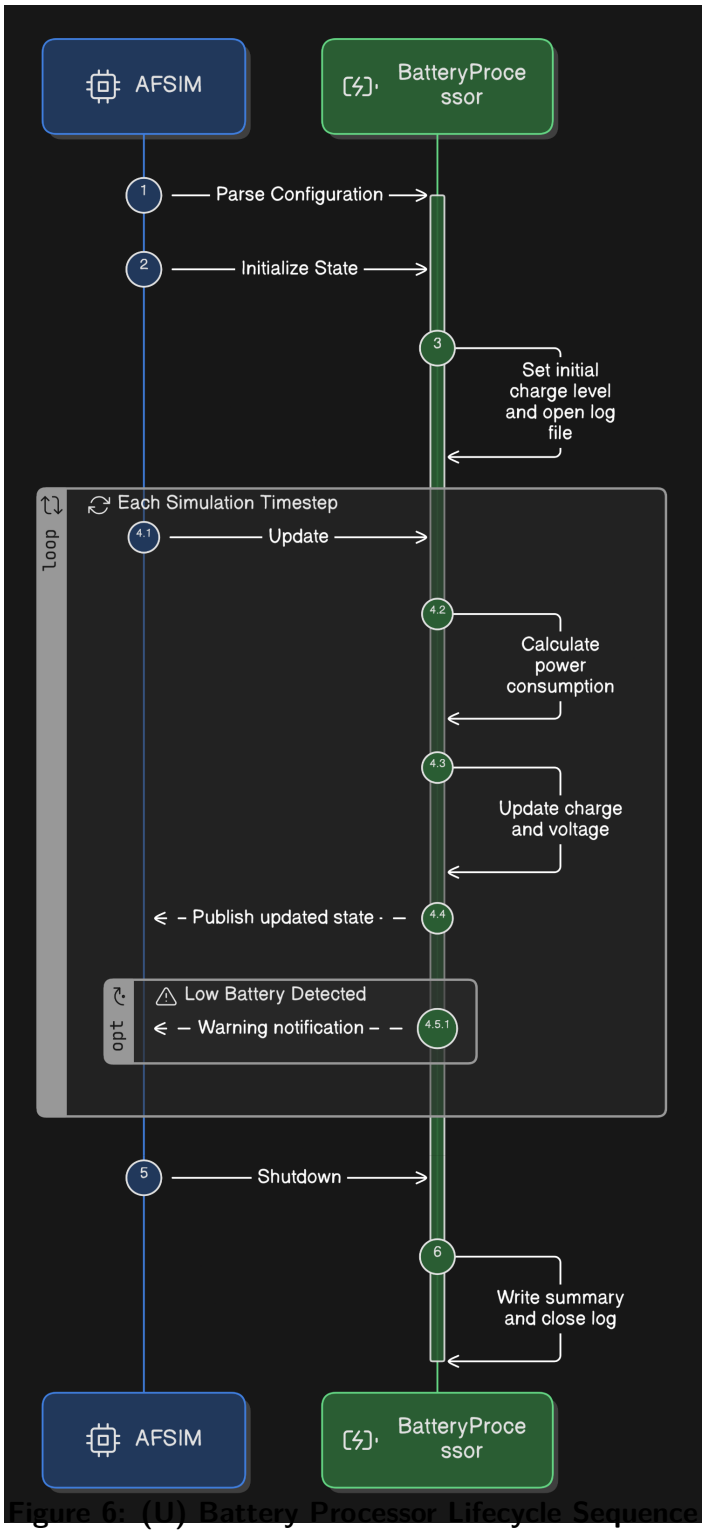


Figure 6: (U) Battery Processor Lifecycle Sequence

4.3 (U) Power Source Resolution

(U) The battery processor reads power consumption through a configurable AuxData key. The resolution order provides fallback behavior:

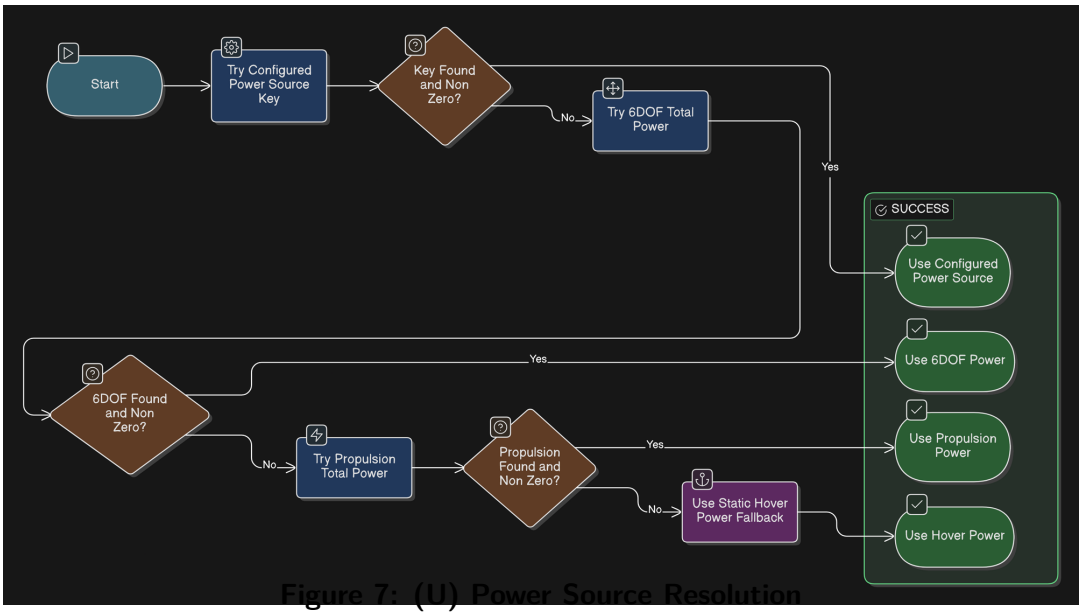


Figure 7: (U) Power Source Resolution

(U) This cascade ensures the battery processor functions regardless of which other plugins are loaded.

4.4 (U) Voltage Sag Solver

(U) The terminal voltage under load is computed by solving for the current that satisfies both:

- $V_{terminal} = V_{oc} - I \cdot R_{internal}$
- $P = V_{terminal} \cdot I$

(U) Substituting yields a quadratic in voltage. The processor selects the physically meaningful root (higher voltage). If the quadratic has no real roots (power demand exceeds what the battery can deliver), the terminal voltage is clamped to the cutoff voltage.

4.5 (U) Public Interface Summary

Table 15: (U) BatteryProcessor Public Interface

UNCLASSIFIED		
Method	Precondition	Expected Outcome
GetSOC()	After Initialize	Returns current state of charge [0–1]
GetVoltage()	After Initialize	Returns terminal voltage [V]
GetCurrent()	After Update	Returns pack current [A]
GetVoltageSag()	After Update	Returns voltage drop from internal resistance [V]
GetRemainingEnergy()	After Update	Returns SOC $\times$ capacity [Wh]
GetCurrentPower()	After Update	Returns power draw this timestep [W]
		UNCLASSIFIED

5 (U) Component: PropulsionProcessor

5.1 (U) Structural Overview

(U) The PropulsionProcessor models the electric motor/ESC/propeller system at a subsystem level. It tracks per-motor state including RPM, current, temperature, and efficiency. It extends WsfProcessor.

(U) **Source files:**

Table 16: (U) Propulsion Source Files

UNCLASSIFIED	
File	Purpose
PropulsionProcessor.hpp	Class declaration, MotorData struct, MotorState enum
PropulsionProcessor.cpp	Implementation
PropulsionRegistration.hpp	Scenario extension
PropulsionPluginRegistration.cpp	Plugin entry points
	UNCLASSIFIED

(U) **Namespace**: t150

5.2 (U) Motor State Array

(U) The processor maintains a fixed-size array of motor states:

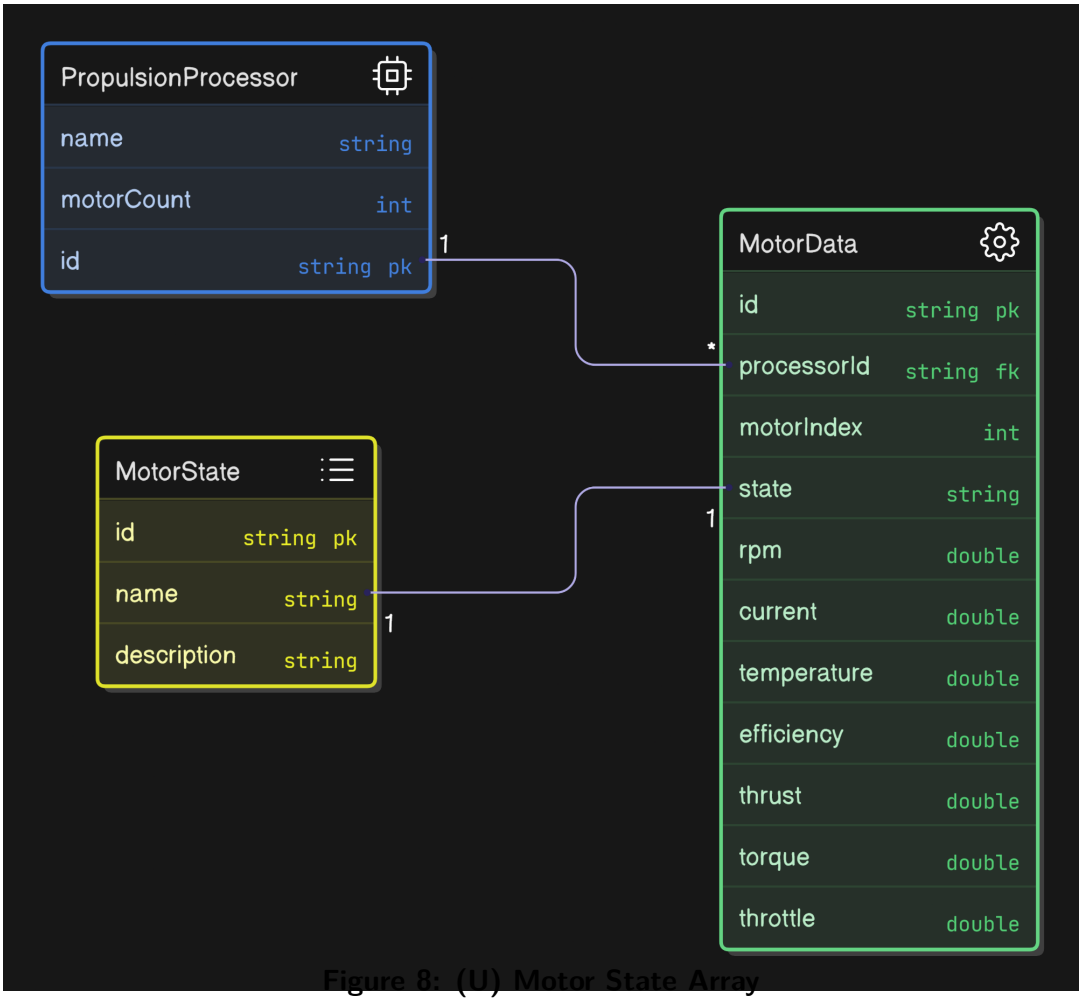


Figure 8: (U) Motor State Array

5.3 (U) Update Loop

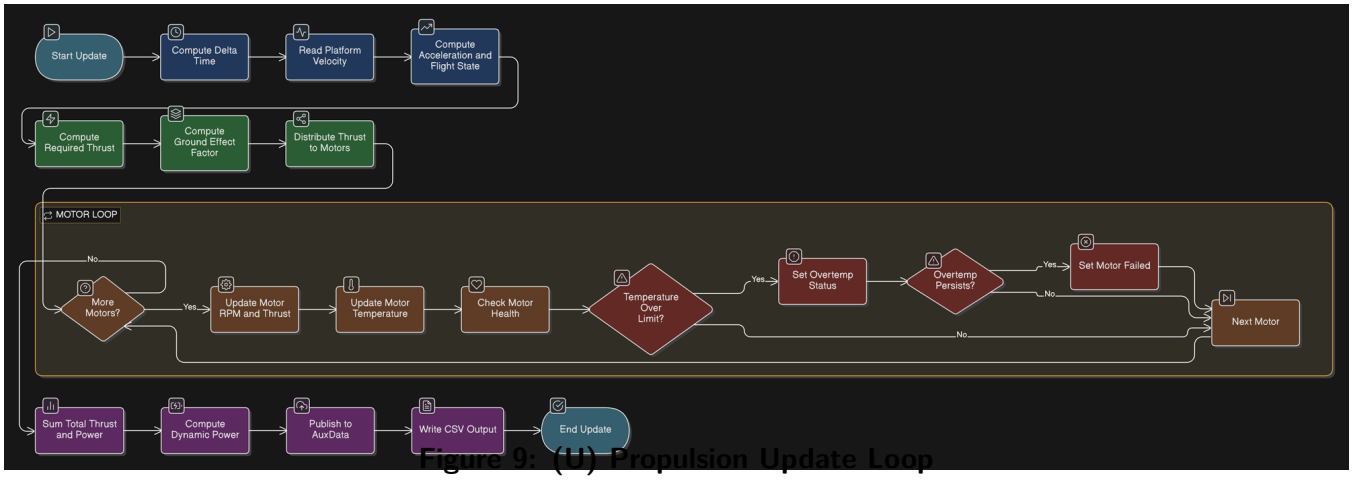


Figure 9: (U) Propulsion Update Loop

5.4 (U) MotorState Machine

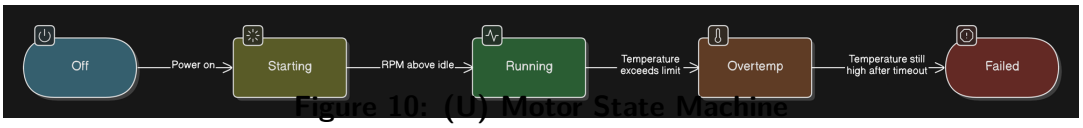


Figure 10: (U) Motor State Machine

(U) Transitions are governed by motor temperature relative to the configurable thermal limit. A failed motor produces zero thrust and is excluded from thrust distribution.

5.5 (U) Public Interface Summary

Table 17: (U) PropulsionProcessor Public Interface

UNCLASSIFIED		
Method	Precondition	Expected Outcome
GetTotalThrust()	After Update	Returns sum of all motor thrust [N]
GetTotalPower()	After Update	Returns total electrical power [W]
GetOperationalMotorCount()	Any time	Returns count of motors not in FAILED state
GetMotorData(index)	$0 \leq$ index $<$ NUM_MOTORS	Returns reference to individual motor state
GetMaxMotorTemperature()	After Update	Returns highest motor temperature [°C]
FailMotor(index)	Motor is RUNNING	Forces motor to FAILED state
GetThrustMargin()	After Update	Returns available thrust margin [0–1]
		UNCLASSIFIED

6 (U) Component: VRSProcessor

6.1 (U) Structural Overview

(U) The VRSProcessor detects and models Vortex Ring State conditions by monitoring descent rate and forward speed. It extends WsfProcessor.

(U) **Source files:**

Table 18: (U) VRS Source Files

UNCLASSIFIED	
File	Purpose
VRSProcessor.hpp	Class declaration, VRSState enum
VRSProcessor.cpp	Implementation
VRSRegistration.hpp	Scenario extension
VRSPluginRegistration.cpp	Plugin entry points
	UNCLASSIFIED

(U) **Namespace**: t150

6.2 (U) VRS State Machine

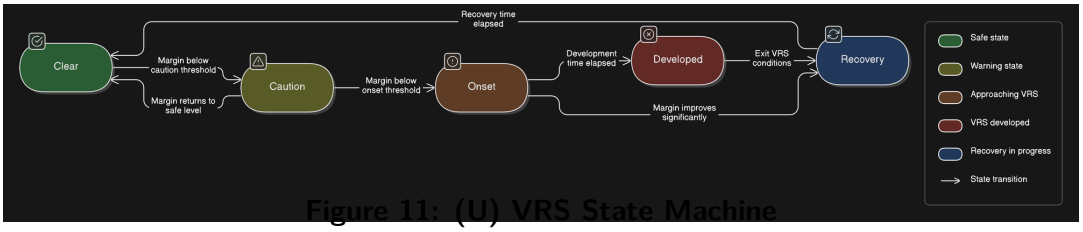


Figure 11: (U) VRS State Machine

Table 19: (U) VRS State Descriptions

UNCLASSIFIED		
State	Condition	Effect
CLEAR	VRS margin $>$ caution threshold	No degradation (lift factor $= 1.0$)
CAUTION	Margin between caution and onset thresholds	Warning issued, no degradation yet
		UNCLASSIFIED

Continued on next page

Table 19: (U) VRS State Descriptions (Continued)

UNCLASSIFIED		
State	Condition	Effect
ONSET	Margin below onset threshold	Partial lift degradation begins
DEVELOPED	Onset sustained for development time	Full lift and control degradation applied
RECOVERY	Conditions improving	Gradual restoration of lift and control
		UNCLASSIFIED

6.3 (U) Velocity Reading

(U) The VRS processor reads descent rate and forward speed from the platform's mover state (via AFSIM's WsfPlatform API), not from AuxData. It calls the platform's velocity accessors to obtain NED velocity components, then derives descent rate (negative of down velocity) and horizontal speed.

6.4 (U) Public Interface Summary

Table 20: (U) VRSProcessor Public Interface

UNCLASSIFIED		
Method	Precondition	Expected Outcome
GetVRSState()	After Update	Returns current VRSState enumeration
GetVRSMargin()	After Update	Returns proximity to VRS [0–1], lower = closer
GetLiftFactor()	After Update	Returns lift multiplier [0–1], 1.0 = no degradation
GetControlFactor()	After Update	Returns control authority [0–1]
IsInVRS()	After Update	Returns true if state == DEVELOPED
GetTimeInState()	After Update	Returns seconds in current state
		UNCLASSIFIED

7 (U) Component: SlungLoadProcessor

7.1 (U) Structural Overview

(U) The SlungLoadProcessor models a cable-suspended payload as a spherical pendulum. It computes swing dynamics, cable tension, and reaction forces that are published to AuxData for the mover to read. It extends WsfProcessor.

(U) **Source files:**

Table 21: (U) Slung Load Source Files

UNCLASSIFIED	
File	Purpose
SlungLoadProcessor.hpp	Class declaration, SlungLoadState enum
SlungLoadProcessor.cpp	Implementation
SlungLoadRegistration.hpp	Scenario extension
SlungLoadPluginRegistration.cpp	Plugin entry points
	UNCLASSIFIED

(U) **Namespace**: t150

7.2 (U) Pendulum Integration Loop

(U) Each timestep, the processor executes:

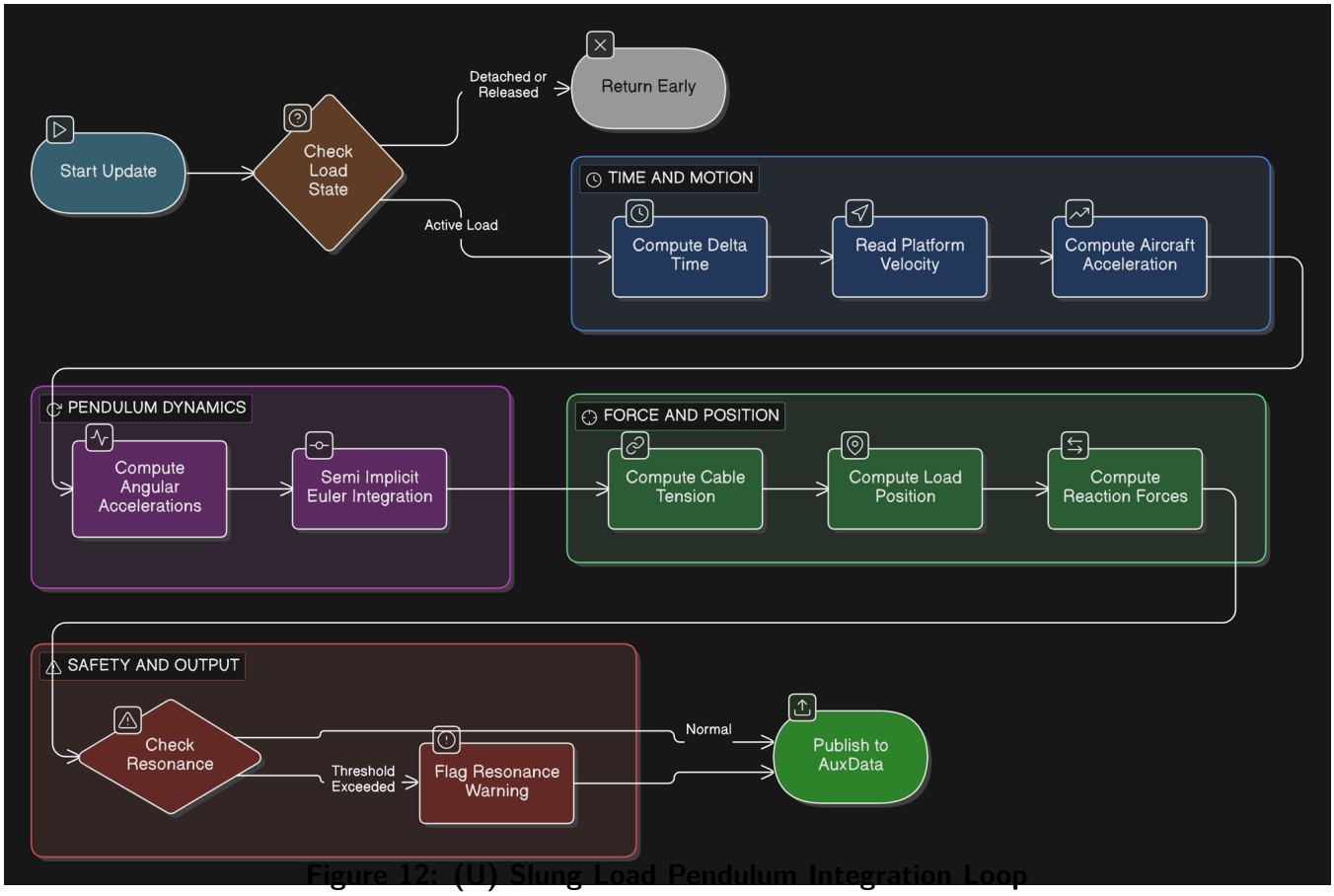


Figure 12: (U) Slung Load Pendulum Integration Loop

7.3 (U) SlungLoad State Machine

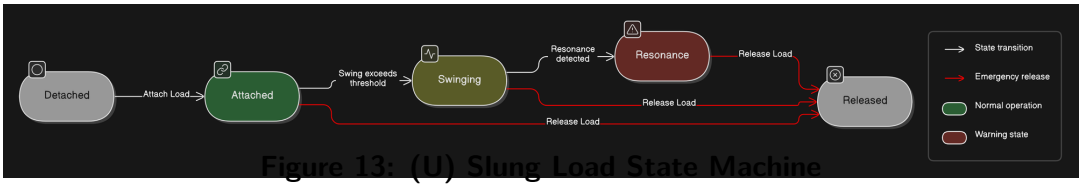


Figure 13: (U) Slung Load State Machine

Table 22: (U) Slung Load State Descriptions

UNCLASSIFIED		
State	Condition	Behavior
DETACHED	No load	No dynamics computed
ATTACHED	Load connected, small swing	Normal pendulum dynamics
SWINGING	Swing angle > threshold	Warning issued, dynamics continue
		UNCLASSIFIED

Continued on next page

Table 22: (U) Slung Load State Descriptions (Continued)

UNCLASSIFIED		
State	Condition	Behavior
RESONANCE	Measured frequency near natural frequency	Critical warning
RELEASED	Load intentionally dropped	Forces zeroed, state frozen
		UNCLASSIFIED

7.4 (U) Force Coupling to Mover

(U) The slung load processor publishes SLUNG_FORCE_X/Y/Z to AuxData. The Rotorcraft6DOF mover reads these keys in its ReadExternalForces() method and adds them to the total force vector before integration. This creates a bidirectional coupling: aircraft motion drives pendulum dynamics (via acceleration), and pendulum forces affect aircraft motion (via AuxData).

7.5 (U) Public Interface Summary

Table 23: (U) SlungLoadProcessor Public Interface

UNCLASSIFIED		
Method	Precondition	Expected Outcome
AttachLoad(mass)	State is DETACHED	Transitions to ATTACHED, sets load mass
ReleaseLoad()	State is ATTACHED/ SWING- ING/RESONANCE	Transitions to RELEASED, zeroes forces
GetSwingMagnitude()	After Update	Returns RSS swing angle [rad]
GetCableTension()	After Update	Returns cable tension [N]
GetLoadForceX/Y/Z()	After Update	Returns reaction force components [N]
IsResonanceWarning()	After Update	Returns true if resonance detected
		UNCLASSIFIED

8 (U) Design Decisions

8.1 (U) Decision Log

Table 24: (U) Design Decision Log

UNCLASSIFIED			
ID	Decision	Rationale	Alternatives Considered
DD-01	Extend WsfRouteMover rather than WsfMover	WsfRouteMover provides built-in route/waypoint management, arrival detection, and pause/resume. Extending WsfMover directly would require reimplementing all route logic.	WsfMover (rejected: too much route logic to reimplement)
DD-02	AuxData blackboard for inter-plugin communication	Decouples plugins; any subset can run independently. Plugins use fallback values when expected AuxData keys are absent.	Direct object references (rejected: build-time coupling); AFSIM message bus (rejected: higher complexity)
DD-03	Quaternion attitude representation	Avoids gimbal lock inherent in Euler angle integration. Quaternion normalization is computationally cheap. Euler angles are derived only for telemetry output.	Euler angles (rejected: gimbal lock at 90-deg pitch); rotation matrices (rejected: 9 elements vs 4)
DD-04	Custom math library (MathUtils.hpp)	AFSIM's built-in math (UtVec3) uses different conventions. A self-contained Vector3/Quaternion/Matrix3x3 library ensures NED consistency and avoids API breakage across AFSIM versions.	UtVec3/UtQuat (rejected: convention mismatch); Eigen (rejected: large dependency for plugin deployment)
DD-05	Per-plugin shared library	Each plugin can be deployed, updated, and tested independently. Scenario files load only the plugins needed.	Monolithic library (rejected: all-or-nothing deployment); header-only (rejected: AFSIM plugins must be shared libraries)
			UNCLASSIFIED

Continued on next page

Table 24: (U) Design Decision Log (Continued)

UNCLASSIFIED			
ID	Decision	Rationale	Alternatives Considered
DD-06	First-order motor lag	Captures motor spin-up/spin-down dynamics with a single time constant parameter. Good fidelity-to-complexity ratio for engagement-level simulation.	Instantaneous response (rejected: unrealistic); second-order dynamics (rejected: added complexity with minimal fidelity gain)
DD-07	Semi-implicit Euler for pendulum integration	Unconditionally stable for the slung load pendulum, simple to implement. Energy dissipation from damping term prevents numerical divergence.	RK4 (considered: more accurate but pendulum damping makes Euler adequate); symplectic Euler (equivalent)
DD-08	Catmull-Rom splines for path interpolation	C1-continuous curves through waypoints without requiring user-specified tangents. Tension parameter provides tuning. Sliding 4-point window keeps memory bounded.	Linear interpolation (rejected: abrupt heading changes); B-splines (rejected: do not pass through control points); Bezier (rejected: require tangent specification)
DD-09	Configurable power source key	Battery processor can read power from any AuxData key, allowing it to pair with the propulsion processor, the 6DOF mover, or a custom source without code changes.	Hard-coded key (rejected: limits reuse); direct processor reference (rejected: coupling)
DD-10	generate_export_header() for DLL support	CMake's GenerateExportHeader module automatically generates the correct __declspec(dllexport) / visibility macros for cross-platform builds.	Manual export macros (rejected: error-prone); WINDOWS_EXPORT_ALL_SYMBOLS only (rejected: fragile with templates)
UNCLASSIFIED			

9 (U) Installation and Build

9.1 (U) Build System

(U) The plugin suite uses CMake (minimum version 3.16) with the following structure:

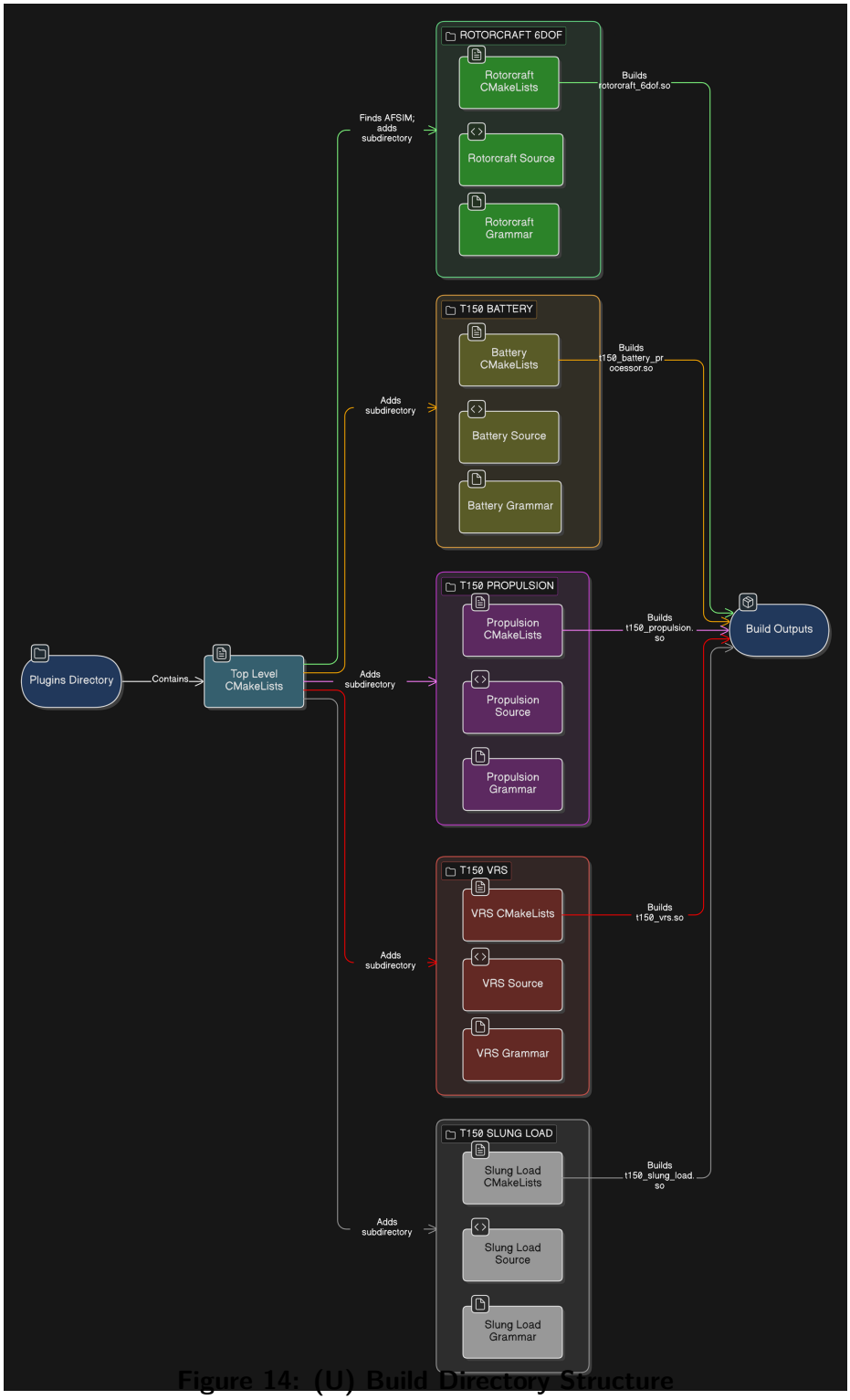


Figure 14: (U) Build Directory Structure

9.2 (U) CMake Configuration

(U) Each plugin's CMakeLists.txt follows the same pattern:

1. (U) Define PLUGIN_NAME
2. (U) Glob source files (*.cpp, *.hpp)
3. (U) Create shared library: add_library(${PLUGIN_NAME} SHARED ...)
4. (U) Generate export header: generate_export_header(${PLUGIN_NAME} ...)
5. (U) Set include directories: source/, build dir, AFSIM includes
6. (U) Link AFSIM::wsf
7. (U) Set properties: POSITION_INDEPENDENT_CODE, PREFIX "", WINDOWS_EXPORT_ALL_SYMBOLS
8. (U) Install to platform-appropriate directory:
   - Linux: lib/wsf_plugins/
   - Windows: bin/mission_plugins/

(U) The top-level CMakeLists.txt detects the AFSIM installation via find_package(afsim REQUIRED) and sets the C++ standard to C++20 (AFSIM 2.2505 requirement).

9.3 (U) Build and Install Scripts

(U) The project provides NGIC convenience scripts at the repository root:

Table 25: (U) Build and Install Scripts

UNCLASSIFIED	
Script	Purpose
ngic_afsim_build.sh	Auto-detects AFSIM installation, runs CMake configure and build
ngic_afsim_install.sh	Installs built plugins to AFSIM system directories (requires sudo)
	UNCLASSIFIED

(U) **Build procedure (pseudocode):**

```
cd plugins/
mkdir -p build && cd build
cmake .. -DAFSIM_ROOT=/opt/afsim2.2505
cmake --build . --parallel
```

(U) **Install destinations:**

Table 26: (U) Install Destinations

UNCLASSIFIED		
Platform	Plugin Directory	Grammar Directory
Linux	/opt/afsim2.2505/lib/wsf_plugins/	/opt/afsim2.2505/bin/grammar/
Windows	C:\AFSIM\afsim2.2505\bin\mission_plugins\	C:\AFSIM\afsim2.2505\bin\grammar\
		UNCLASSIFIED

9.4 (U) Compiler Requirements

Table 27: (U) Compiler Requirements

UNCLASSIFIED		
Compiler	Minimum Version	Platform
GCC	13.3	Linux (AlmaLinux 9 WSL)
MSVC	2022 (v143)	Windows 11
CMake	3.16	Both
C++ Standard	C++17 (CMake configured for C++20)	Both
		UNCLASSIFIED

10 (U) Sources and References

(U) Reference the UAM for a comprehensive list of sources used to develop this model. The following references were used in creating this ADD.

Table 28: (U) References

UNCLASSIFIED	
Ref	Citation
[1]	Department of Defense Instruction 5000.61, "DoD Modeling and Simulation (M&S) Verification, Validation, and Accreditation (VV&A)," current version.
[2]	MIL-STD-3022, "Documentation of Verification, Validation, and Accreditation (VV&A) for Models and Simulations," current version.
[3]	T-150 UAS AFSIM Plugin Suite – Underlying Algorithms Model (UAM), Version 3.0, March 2026.
[4]	AFSIM User's Guide, Version 2.2505, Air Force Research Laboratory, 2025.
[5]	Johnson, W. *Helicopter Theory*, Dover Publications, 1994.
[6]	Leishman, J. G. *Principles of Helicopter Aerodynamics*, 2nd ed., Cambridge University Press, 2006.
[7]	Brand, A. et al. "Vortex Ring State (VRS) Prediction," AIAA Paper, 2004.
[8]	Cicolani, L. S. et al. "Simulation of the Dynamics of Helicopter Slung Load Systems," NASA/TP-2009-215656, 2009.
[9]	Tremblay, O. and Bhatt, A. "Experimental Validation of a Battery Dynamic Model for EV Applications," *World Electric Vehicle Journal*, Vol. 3, 2009.
[10]	T-150 UAS System Specification, [PLACEHOLDER – Document ID], current version.
[11]	AFSIM Plugin Developer's Guide, Air Force Research Laboratory, 2025.
	UNCLASSIFIED

11 (U) Acronyms

Table 29: (U) Acronyms

UNCLASSIFIED	
Acronym	Definition
ADD	Architecture Description Document
AFSIM	Advanced Framework for Simulation, Integration, and Modeling
AGL	Above Ground Level
API	Application Programming Interface
CG	Center of Gravity
CMake	Cross-platform build system
CW/CCW	Clockwise / Counter-Clockwise
DLL	Dynamic Link Library
DoD	Department of Defense
DoDI	Department of Defense Instruction
DOF	Degrees of Freedom
ESC	Electronic Speed Controller
GCC	GNU Compiler Collection
GE	Ground Effect
ISA	International Standard Atmosphere
LOC	Lines of Code
MSL	Mean Sea Level
MSVC	Microsoft Visual C++
NED	North-East-Down (coordinate frame)
NGIC	National Ground Intelligence Center
OCV	Open-Circuit Voltage
PID	Proportional-Integral-Derivative (controller)
RPM	Revolutions Per Minute
SOC	State of Charge
UAS	Unmanned Aircraft System
UAM	Underlying Algorithms Model
UML	Unified Modeling Language
VRS	Vortex Ring State
VV&A	Verification, Validation, and Accreditation
	UNCLASSIFIED

Continued on next page

Table 29: (U) Acronyms (Continued)

UNCLASSIFIED	
Acronym	Definition
WSF	Warfighter Simulation Framework
	UNCLASSIFIED